Спеціальні розділи обчислювальної математики
Комп'ютерний практикум №1

Багаторозрядна арифметика

ФІ-43 Редько Микита

9 жовтня 2026 р.

1 Мета роботи

Отримання практичних навичок програмної реалізації багаторозрядної арифметики; ознайомлення з прийомами ефективної реалізації критичних по часу ділянок програмного коду та методами оцінки їх ефективності.

2 Результати

Нижче наведено тести для основних операцій багаторозрядної арифметики. Їх успішне виконання засвідчує коректність нашої імлементації.

2.1 Перетворення чисел

Перевіряються перетворення малих констант (u64), перетворення рядка з шістнадцятковим числом та зворотне перетворення числа у рядок.

```
#[test]
fn test_u64_to_big_int_zero() {
    let expected = BigInt {
        limbs: [0; BIG_INT_LIMBS],
    };

    assert_eq!(u64_to_big_int(0), expected);
}

#[test]
fn test_u64_to_big_int_value() {
    let mut limbs = [0; BIG_INT_LIMBS];
    limbs[0] = 91726;
    let expected = BigInt { limbs };

    assert_eq!(u64_to_big_int(91726), expected);
}

#[test]
fn test_hex_to_big_int_multiple() {
    let value = hex_to_big_int("1000000000000123A").unwrap();

    assert_eq!(big_int_to_hex(&value), "1000000000000123A");
}

#[test]
```

```rust
fn test_big_int_to_hex_single() {
    let mut limbs = [0u64; BIG_INT_LIMBS];
    limbs[0] = 0x123A;
    let number = BigInt { limbs };

    assert_eq!(big_int_to_hex(&number), "123A");
}
```

2.2 Порівняння, додавання та віднімання

Тести перевіряють результати порівняння, перенесення під час додавання, віднімання з запозиченням та випадок, коли результат віднімання є від'ємним.

```rust
#[test]
fn test_cmp_equal() {
    let a = u64_to_big_int(7);
    let b = u64_to_big_int(7);

    assert!(matches!(compare_big_int(&a, &b), Comparison::Equal));
}

#[test]
fn test_cmp_less() {
    assert!(matches!(
        compare_big_int(&u64_to_big_int(6), &u64_to_big_int(7)),
        Comparison::Less
    ));
}

#[test]
fn test_cmp_greater() {
    assert!(matches!(
        compare_big_int(&u64_to_big_int(7), &u64_to_big_int(6)),
        Comparison::Greater
    ));
}

#[test]
fn test_add_big_int_carry() {
    let mut a_limbs = [0; BIG_INT_LIMBS];
    a_limbs[0] = u64::MAX;
    a_limbs[1] = 4;
    let a = BigInt { limbs: a_limbs };

    let mut b_limbs = [0; BIG_INT_LIMBS];
    b_limbs[0] = 1;
    b_limbs[1] = 2;
    let b = BigInt { limbs: b_limbs };

    let mut expected_limbs = [0; BIG_INT_LIMBS];
    expected_limbs[1] = 7;
    let expected = BigInt {
        limbs: expected_limbs,
    };

    assert_eq!(add_big_int(&a, &b), expected);
}

#[test]
fn test_subtract_success() {
    let a = hex_to_big_int("10000000000000000").unwrap();
    let b = u64_to_big_int(1);
```

```
    let expected = hex_to_big_int("FFFFFFFFFFFFFFFF").unwrap();

    assert_eq!(subtract_big_int(&a, &b), Some(expected));
}

#[test]
fn test_subtract_fail() {
    assert_eq!(
        subtract_big_int(&u64_to_big_int(1), &u64_to_big_int(2)),
        None
    );
}
```

2.3 Множення та піднесення до квадрата

Перевіряються множення двох багаторозрядних чисел, дистрибутивність множення відносно додавання, множення як повторне додавання, результат піднесення до квадрата та узгодженість квадрата з множенням числа на себе.

```
#[test]
fn test_mult_big_int() {
    assert_eq!(
        mult_big_int(
            &hex_to_big_int("ABCDEFEFFDFDDBCBDBAAAAACEEEEEEFEEFE").unwrap(),
            &hex_to_big_int("BABFAFBAEBAFEAFEAEF").unwrap()
        ),
        hex_to_big_int(
            "7D5444E99B6A6FD5AAA81359C0B5FFAC76C7A80AD34528C6164B22"
        )
        .unwrap()
    );
}

#[test]
fn test_mult_distributive_over_addition() {
    let a = hex_to_big_int("FEDCBA9876543210FFFFFFFFFFFFFFFF").unwrap();
    let b = hex_to_big_int("123456789ABCDEF0123456789ABCDEF0").unwrap();
    let c = hex_to_big_int("10000000000000001").unwrap();
    let sum = add_big_int(&a, &b);

    assert_eq!(
        mult_big_int(&sum, &c),
        add_big_int(&mult_big_int(&a, &c), &mult_big_int(&b, &c))
    );
}

#[test]
fn test_mult_by_repeated_addition() {
    let multiplier = u64_to_big_int(100);
    let value = hex_to_big_int("FFFFFFFFFFFFFFFF0000000000000001").unwrap();
    let mut sum = u64_to_big_int(0);

    for _ in 0..100 {
        sum = add_big_int(&sum, &value);
    }

    assert_eq!(mult_big_int(&multiplier, &value), sum);
}

#[test]
fn test_square_big_int() {
    let value = hex_to_big_int("ABCDEFEFFDFDDBCBDBAAAAACEEEEEEFEEFE").unwrap();
```

```
    let expected = hex_to_big_int(concat!(
        "734CC434C48FE970985A9A52515963D6E4643A",
        "2B2A625BDE8BBCB60B1C298865614404"
    ))
    .unwrap();

    assert_eq!(square_big_int(&value), expected);
}

#[test]
fn test_square_mult_consistency() {
    let value = hex_to_big_int("ABCDEFEFFDFDDBCBDBAAAAACEEEEEEFEEFE").unwrap();

    assert_eq!(square_big_int(&value), mult_big_int(&value, &value));
}
```

2.4 Ділення та остача

Окремо перевіряється ділення на нуль, ділення меншого числа на більше, більшого на менше. Для останніх двох випадків тест порівнює і частку, і остачу.

```
#[test]
fn test_divide_mod_by_zero() {
    assert_eq!(
        divide_mod_big_int(&u64_to_big_int(10), &u64_to_big_int(0)),
        None
    );
}

#[test]
fn test_divide_mod_larger_by_smaller() {
    let a = hex_to_big_int(concat!(
        "AABABABABABABABABABABABAABABABABAAABABABABAB",
        "ABABABABABABABABABABABAAABABAB"
    ))
    .unwrap();
    let b = hex_to_big_int("342345346453453245345646353456435634563434563").unwrap();

    assert_eq!(
        divide_mod_big_int(&a, &b),
        Some((
            hex_to_big_int("3464AFF4F01C1CA3B3320E2D812").unwrap(),
            hex_to_big_int("1C29AEB03271A65F72AF3C44D7B4CEE6937E9AEB6D9F").unwrap()
        ))
    );
}

#[test]
fn test_divide_mod_smaller_by_larger() {
    let a = hex_to_big_int("342345346453453245345646353456435634563434563").unwrap();
    let b = hex_to_big_int(concat!(
        "AABABABABABABABABABABABAABABABABAAABABABABAB",
        "ABABABABABABABABABABABAAABABAB"
    ))
    .unwrap();

    assert_eq!(divide_mod_big_int(&a, &b), Some((u64_to_big_int(0), a)));
}
```

2.5 Піднесення до степеня

Тест звіряє результат піднесення багаторозрядного числа до степеня з наперед обчисленим значенням.

```
#[test]
fn test_horner_exponentiation() {
    let p = hex_to_big_int(
        "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F"
    )
    .unwrap();
    let exponent = u64_to_big_int(7);
    let expected = concat!(
        "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF",
        "FFFFFFFFFFFFFFF8FFFFE5490000000000000000",
        "00000000000000000000000000000150000A04A",
        "0131DD34FFFFFFFFFFFFFFFFFFFFFFFFFFFFFF",
        "FFFFFFDCFFFE6F46FA06ADEF667F4E8D00000000",
        "000000000000000000000230002164C0BF2A430",
        "6602E2CCD42537E2FFFFFFFFFFFFFFFFFFFFFEA",
        "FFFE6F46F40D5BC066FB804B834DEEA46B936D9B",
        "00000000000000070000A04A05F9522766031CCE",
        "7CF47B0E7D559AF77B739126FFFFFFFEFFFFE548",
        "FECE22C3667F318C2B985E6A1716F73F74B12531",
        "8D22B84F"
    );

    assert_eq!(
        horner_exponentiation_big_int(&p, &exponent),
        hex_to_big_int(expected).unwrap()
    );
}
```

3 Підтвердження проходження тестів

Вище наведено результат запуску cargo test: усі 29 тестів пройшли успішно.

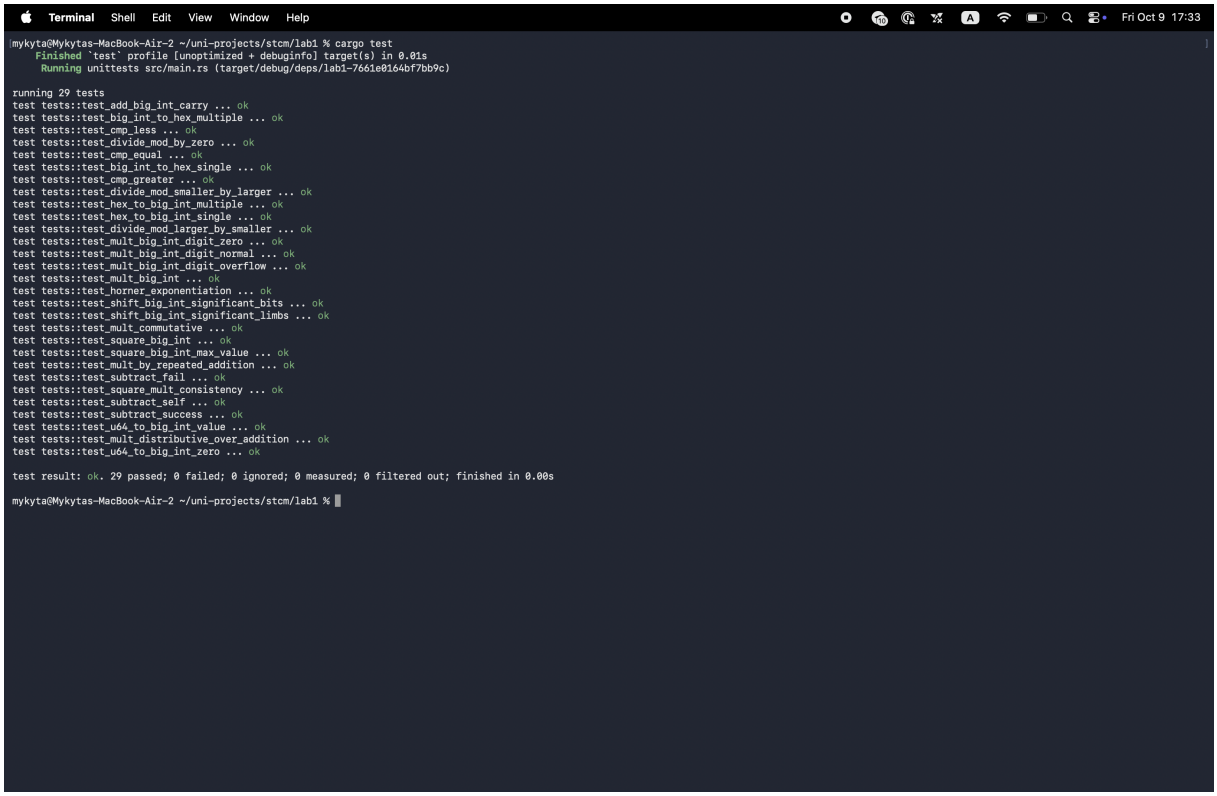


Рис. 1: Успішне виконання тестів

3.1 Результати бенчмарків

Для кожної операції виконано 1000 прогрівальних запусків, після чого виміряно середній час для 10000 наборів випадкових вхідних даних. Числа представлені. Показник часу наведено в наносекундах на одну операцію. Результати отримано під час запуску оптимізованої release-збірки командою cargo run –release.
Додавання 78.9 нс/операцію

Віднімання 52.6 нс/операцію

Порівняння 4.0 нс/операцію

Множення 2358.6 нс/операцію

Піднесення до квадрата 626.3 нс/операцію

Ділення та знаходження остачі 104.5 нс/операцію

Піднесення до степеня методом Горнера 10147.0 нс/операцію